Section 06: React 基礎

Web 画面の仕組みと React の登場

UI を「部品」として管理し、効率的に画面を構築する手法を学びます。

0. 環境構築 (Vite + React)

React の学習を始める前に、手元で動かせる環境を作りましょう。
まずは土台となる **Node.js (npm)** とビルドツール **Vite** について解説します。

Step 0: Vite（ヴィート）とは？

Vite は、一言でいえば **「超高速なプロジェクト作成＆開発ツール」** です。

従来のツール（Create React App など）は、ファイル数が増えると起動やプレビューの更新に数秒〜数十秒かかることがありました。
Vite は最新のブラウザ機能を活用することで、規模が大きくなっても **一瞬でサーバーが起動し、変更が即座に画面に反映** されます。

Step 1: プロジェクトの作成

ターミナルを開き、Vite を使って雛形（テンプレート）を作成します。

```
npm create vite@latest my-react-app
```

Step 2: 依存パッケージのインストール

作成されたフォルダに入業し、実行に必要なパッケージ一式をダウンロードします。

```
cd my-react-app
npm install
```

Step 3: 開発サーバーの起動

インストールが終わったら、プレビュー用の開発サーバーを立ち上げます。

```
npm run dev
```

起動後、ターミナルに表示される URL (例: `http://localhost:5173/`) にブラウザでアクセスし、画面が表示されれば準備完了です！

0. Web 画面が構成される仕組み

ブラウザはどのようにWeb画面を表示しているのでしょうか？

そもそも「ブラウザ」とは？

いつも Google や YouTube を見るときに使っている **Chrome** や **Safari** などのソフトのことです。

プログラミングの視点で見ると、ブラウザは **「コード（テキスト）を読み込んで、私たちが操作できる『画面』へと翻訳・組み立てをしてくれる専用ソフト」** です。

ブラウザの主な仕事

ブラウザは画面を表示するために、内部で 3 つの大きな仕事をしています。

1. **取得**: サーバー（ネットワーク）から、HTML などのファイルを貰ってくる
2. **描画（レンダリング）**: HTML や CSS を読み取って、パズルのように画面を組み立てる
3. **実行**: JavaScript を読み込んで、ボタンを押したときの動きなどを実現する

そもそも HTML とは？

HTML (HyperText Markup Language) は、**「文章の構造」** をコンピュータに伝えるための言語です。

「ここからここまでは **見出し**」「ここは **段落**」「ここは **画像**」といった情報を、**タグ** という記号で囲って表現します。

```
<h1>タイトル</h1>
<p>これは本文です。</p>
```

HTML の基本ルール：タグ

HTML は基本的に **「開始タグ」** と **「終了タグ」** で中身を挟みます。

- 挟まれた部分が、そのタグの意味（役割）を持ちます。
- **入れ子構造**: タグの中にさらにタグを入れることで、複雑な構造（リストや表など）を作ります。

三種の神器：HTML / CSS / JavaScript

モダンな Web 開発でも、最終的にブラウザが解釈するのはこの 3 つです。

- **HTML (骨組み)**: どこに「見出し」があり、どこに「ボタン」があるかという構造を定義します。
- **CSS (装飾)**: 色、サイズ、余白、アニメーションなど、見た目を整えます。
- **JavaScript (振る舞い)**: 「ボタンを押したらメニューが開く」「データを読み込む」といった動的な動きを担当します。

DOM (Document Object Model)

ブラウザが HTML を読み込むと、内部で **「DOM」** という木の枝のような形をしたデータに変換します。

JavaScript で画面を書き換える＝ **「DOM というデータを直接いじって書き換える」** という作業になります。

```
// 古典的な「DOM操作」の例
const button = document.getElementById("my-btn");
// ボタンを書き換える「命令」を一つずつ書く必要がある
button.innerText = "クリックされました！";
button.style.backgroundColor = "gold";
```

「命令（どうやるか）」から「宣言（どうあるべきか）」へ

手作業で DOM をいじる（Vanilla JS）のは、画面が複雑になると「どこで何を書き換えたか」が分からなくなり、バグの温床になります。

- **Vanilla JS (命令型)**: 「ボタンを探して、赤くして、文字を変えて...」
- **React (宣言型)**: **「状態が『押された』なら、このボタンは赤くて、文字はこの通りである」** と定義するだけ。

「手順」ではなく「結果の状態」を書くのが、React の最大の特徴です。

1. なぜ React を使うのか？

これまでの「画面更新」の大変さ

昔ながらの JavaScript (`Vanilla JS`) では、画面を少し変えるだけでも手順が複雑でした。

例：ボタンのテキストとスタイルを変える場合

1. ボタンの要素を探す (`document.getElementById`)
2. その中身のテキストを書き換える (`innerText`)
3. スタイルを変える(`style.backgroundColor`)

昔ながらの JavaScript (`Vanilla JS`)は、あくまで「ここをこう変えなさい」の集合であり、「現在どうなっているのか」を把握するのは少し難しい。

「今の画面がどうなっているか」を常にプログラミングで追いかけるのは、限界がありました。

React：宣言的な UI

React は **「データがこうなれば、画面はこうなる」** というルールを定義するだけです。

- **DOM操作はReactエンジンにお任せ**: 直接「画面を書き換えろ」と命令する必要はありません。
- **コンポーネント指向**: 小さな部品（ボタン、入力欄など）を組み合わせて巨大な画面を作ります。
- **高速な更新**: 「必要な部分だけ」を賢く書き換える仕組み（仮想DOM）を持っています。

2. コンポーネント：UI の最小単位

React Component Tree

React では、UI の部品を JavaScript/TypeScript の **「関数」** として定義します。

```
const WelcomeMessage = () => {
  return (
    <div className="card">
      <h1>こんにちは！</h1>
      <p>GDGoCへようこそ。</p>
    </div>
  );
};
```

コンポーネントは「レゴブロック」

React の開発は、大きな画面をいきなり作るのではなく、**小さな部品（コンポーネント）を作って組み合わせる**作業です。

- **再利用ができる**: 一度「ボタン」や「カード」を作れば、何回でも使い回せます。
- **管理が楽**: 「タイトルがおかしい」と思ったら、タイトルのコンポーネントだけを直せば OK です。
- **組み合わせる**: 小さな「ボタン」や「入力欄」を組み合わせて、「ログインフォーム」という大きな部品を作れます。

コンポーネント化：コードの見え方はどう変わる？

お城の「図面」を別の図面の中で呼び出すイメージです。
どちらも **ブラウザに描画される結果は全く同じ** です。

A: 全部そのまま書く（親の中に生 TSX）

```
const App = () => {
  return (
    <main>
      <div className="card">
        <h1>こんにちは！</h1>
        <p>GDGoCへようこそ</p>
      </div>
    </main>
  );
};
```

B: 子コンポーネントを呼び出す

```
const App = () => {
  return (
    <main>
      {/* 名前をつけて部品を呼び出す */}
      <WelcomeMessage />
    </main>
  );
};
```

TSX：TypeScript で UI を書くための「HTMLの拡張」

コンポーネントの `return` の中に書かれている、HTML のようなコードが **TSX** です。

```
const WelcomeMessage = () => {
  return (
    <div className="card">
      <h1>こんにちは！</h1>
      <p>GDGoCへようこそ。</p>
    </div>
  );
};
```

1. **見た目は HTML**: `<div>`, `<h1>` など、見慣れたタグがそのまま使えます。
2. **中身は JavaScript**: `{ }` （波括弧）を使うと、その中で変数や計算式を自由に書けます。
3. **型の恩恵**: 変数名の間違いや、型が合わないときは、書いている最中にエディタが教えてくれます。

Props：部品に「注文」を付ける

同じ「スタンプ（コンポーネント）」でも、押すたびに中身のテキストや色だけを変えたい場合があります。
そのための「カスタム注文」の仕組みが **Props（プロップス）** です。

お城づくりでいえば、「青いドア」「赤いドア」といった、**図面への細かい指定**に相当します。

```
// 部品側
const UserCard = ({ name, role }: { name: string; role: string }) => {
  return (
    <div>
      <h3>{name}</h3>
      <p>{role}</p>
    </div>
  );
};

// 使う側
const App = () => {
  return (
    <>
      <UserCard name="たなか" role="部長" />
      <UserCard name="さとう" role="エンジニア" />
    </>
  );
};
```

3. State & Event：画面に「動き」をつける

useState：画面を動かすための「特別な記憶」

関数の中身（コンポーネント）は、画面が書き換わる（再描画される）たびに、毎回「最初から最後まで」実行し直されます。
普通の変数（`let` など）だと、そのたびに値がリセットされて消えてしまいます。

これを防ぎ、値を「記憶」し続けるための道具が `useState` です。

- **「消えない記憶」**：画面が書き換わっても、Reactが値を預かっておいてくれます。
- **「更新の合図」**：値が変わった瞬間に、Reactへ「画面を書き換えて！」とベルを鳴らしてくれます。

useState の書き方

```
import { useState } from "react";

const Counter = () => {
  // const [今の値, 値を書き換えるための関数] = useState(最初の値);
  const [count, setCount] = useState(0);

  return (
    <div>
      <p>現在のカウント: {count}</p>
      <button onClick={() => setCount(count + 1)}>カウントアップ！</button>
    </div>
  );
};
```

イベントハンドリング：`onClick` や `onChange`

ユーザーの操作を受け取るには、タグに `on∞` という属性を付けます。
=の先につくのは、その操作が行われたときに実行したい関数です。

- `onClick`：クリックされた時
- `onChange`：入力内容が変わった時（`<input>` など）

```
// 入力内容をリアルタイムで State に入れる例
<input type="text" onChange={(e) => setName(e.target.value)} />
```

4. 副作用（Side Effect）とは何か？

ここからは、React 中級者への第一歩である「副作用」について学びます。

React の「メインのお仕事」

React コンポーネントには、本来 **たった1つしか仕事がありません。**
それは **「Props や State を元に、画面（HTML）を計算して返すこと」** です。

1. `count` が `0` だから、`0` と表示する画面を作る。
2. `count` が `1` になったら、`1` と表示する画面を作り直す。

この「画面を作る計算処理」のことを、専門用語で **レンダリング** と呼びます。

メインの仕事中に「別の重い仕事」を頼むと？

しかし、実際の Web アプリでは「画面を作る」以外にもやりたいことがたくさん出てきます。

- サーバーからデータを取ってくる（通信）
- ブラウザのタブのタイトルを変更する
- タイマーをセットする

もし、React が「画面を作る計算」をしている真っ最中に、「ちょっと通信してくるね（数秒かかる）」と**寄り道**をしてしまったらどうなるでしょうか？

画面がフリーズしてしまう！

計算中に寄り道をすると、**「画面を描画する作業」が途中でストップ** してしまいます。
ユーザーから見ると「ボタンを押したのに、画面が固まって動かない」という最悪の開発体験になります。

React の世界では、このように「画面（UI）を作る計算以外の、外部と関わる処理」のことを **「副作用 (Side Effect)」** と呼びます。

副作用のお願いは「useEffect」へ

「画面作り」を邪魔せずに、安全に副作用を行いたい。
そのために用意されているのが `useEffect` です。

`useEffect` は、React に対する **「後回しのお願い箱」** です。
「今すぐやらなくていいから、**画面をユーザーに見せ終わって一段落したら**、これやっておいて！」と頼むことができます。

useEffect の基本形式

```
import { useEffect } from "react";

useEffect(
  () => {
    // ① ここに「やりたいこと（副作用）」を書く
    console.log("画面が出た後に実行されたよ！");
  },
  [
    /* ② ここに「いつ実行するか」のルールを書く */
  ],
);
```

- **第1引数**: 実行したい関数を書きます。
- **第2引数**: **依存配列** と呼びます。実行するタイミングをコントロールする「ルール」を入れます。

依存配列 `[]` のパターン（超重要）

`useEffect` は、いつ実行するかをコントロールするために「依存配列」を使います。

1. **`[]` （空の配列）**
   - **「画面が最初に表示された時の 1 回だけ」** 実行されます。（例：初回アクセス時にデータを1回だけ取得する）
2. **`[変数]`**
   - 指定した変数が **「変化した時だけ」** 実行されます。（例：数字が変わったらタイトルを変える）
3. **（配列を書かない）**
   - 画面が描画されるたびに **「毎回」** 実行されます。無限ループになりやすいので注意しましょう。

【実践】useEffect を書いてみよう

```
import { useState, useEffect } from "react";

const TitleApp = () => {
  const [count, setCount] = useState(0);

  // 画面の表示が終わった「後」に実行される
  useEffect(() => {
    document.title = `現在のカウント: ${count}`;
    console.log("タイトルを更新しました！");
  }, [count]); // count が変わった時だけ実行

  return <button onClick={() => setCount(count + 1)}>押した回数: {count}</button>;
};
```

結局 useEffect は「いつ」「どう」使えばいいのか

- **いつ使う？**
  画面の表示・更新に「つられて」実行したい **副作用（タイトル変更・API 呼び出し・保存など）** があるとき。
- **どう使う？**
  「画面がどう描かれるか」に必要なロジックは `useEffect` の外、「描かれたあとに起こる処理」は `useEffect` の中に書く。
- **依存配列の決め方**
  「**何が変わったときにこの処理を走らせたいか**」だけを `[]` / `[count]` / `[userId]` のように列挙する。

5. 【ワークショップ】電卓アプリを作ろう

学んだ知識（Components, Props, State, Events）を総動員して、実際に動く電卓を1から組み立てます。

Step 1: 電卓の「数字」を表示する (State)

まずは、今どの数字が入力されているかを記憶する `useState` を用意します。

- **ヒント**
  - `const [display, setDisplay] = useState("0");` のような State を用意する
  - JSX に「表示エリア」として `display` を表示する `div` を1つ作る

Step 2: 数字ボタンを作る (Events)

数字が押されたら、現在の表示に追加する関数を作ります。

- **ヒント**
  - 引数で受け取った数字（`"1"`, `"2"` など）を、`display` の末尾にくっつける関数 `handleNumber(num: string)` を作る
  - `display` が `"0"` のときだけ、上書きするようにすると使いやすい
  - ボタンは `onClick={() => handleNumber("1")}` のようにしてイベントをつなぐ

Step 3: 計算を実行する (Logic)

「AC」ボタンでクリア、「=」ボタンで計算を実行するロジックを作ります。

- **ヒント**
  - 「12+3-4」のような文字列を 1 文字ずつ読みながら、`["12", "+", "3", "-", "4"]` のような配列に分解する
  - 配列の **先頭を初期値** にして、その後ろを「演算子」「数値」「演算子」「数値」…の順で左からなめて計算する
  - 今回はシンプルに `+` **と** `-` **だけ** に対応すれば OK

Step 4: 完成像のコード例

- **ここまでの Step 1〜3 を組み合わせて 1 つのコンポーネントにまとめる**
- 「表示エリア」「数字ボタン」「`AC` ボタン」「`=` ボタン」がそろっていれば OK
- 完成例のコードは、リポジトリ内の `Attendee-resources/part6/calculator-sample.tsx` に用意してあります（詰まったら参照して OK）

Part 06 のまとめ

1. **コンポーネント**: UIを「図面」や「部品」として分ける。
2. **Props**: 部品に外から「設定」や「注文」を渡す。
3. **useState**: 画面に「記憶」を持たせ、動的に更新する。
4. **useEffect**: 重い処理や画面外の処理を「後回し」にする。

お疲れ様でした！React の基本はマスターです。次からは、これらを AI やバックエンドと繋ぐ方法を学んでいきましょう！